AI NATIVE CAMP 1기 I DAY 4

만든 걸
내 것으로 만드는 법

철학과 논리 — Compound Engineering

장원준 (zoon)

2026. 2. 18 I AI Native Camp 1기 Day 4

Day 1: "왜 여기 모였는가"

- 200명 → 45명. 이 방의 밀도가 전부
- 일하는 방식이 먼저, 도구는 결과
- Claude Code 환경 세팅 + 첫 체험

가져간 것

첫 CLAUDE.md

"그날 CLAUDE.md를 처음 만들었습니다. 그게 시작이었습니다."

Day 2: "도구에는 설계 철학이 있다"

- 6개 도구의 KPI와 설계 철학
- MCP 연결: 도구들이 Claude와 대화 시작
- 패널: 안철우(커뮤니티), 최은영(CRM)

가져간 것

MCP 연결 환경

"도구마다 다른 맥락이 있다는 걸 배웠고, 그 맥락을 AI에게 연결했습니다."

Day 3: "실제로 돌아가는 걸 봤다"

- 라이브 팟캐스트: AI 시대의 VC
- 운영 자동화 라이브 시연

  슬랙 수집 → Clarify → 과제 자동 생성

가져간 것

스킬 제작 경험

"어제는 제가 철학 얘기를 안 했습니다. 그 이유를 지금부터 설명합니다."

Day 4: "만든 걸 내 것으로 만드는 법"

CLAUDE.md ─┐

MCP 연결 ──┼→ 쓰고 → 고치고 →

쌓는다

스킬 ──┘

Compound Engineering

"4일 동안 세 가지를 만들었습니다. 오늘은 이것들을 내 마음에 들게 만드는 법."

왜 이 순서로 갔는가

Day 1에서 했던 말

Stage 1 AI에게 잘 물어보기

Stage 2 AI가 내 맥락을 이해하게 만들기

Stage 3 AI와 함께 일하는 시스템을 만들기

커리큘럼 매핑

Day 1-2 = Stage 1→2 환경 세팅

Day 3 = Stage 2→3 실행 체험

Day 4 = Stage 3을 유지하는 방법 ← 오늘

Day 5-7 = 자기 것으로 완성

업무 운영체제(OS)라는 비유

컴퓨터의 OS

하드웨어를 제어하고, 앱을 실행하고, 파일을 관리한다

일할 때의 OS

도구를 제어하고, 워크플로우를 실행하고, 맥락을 관리한다

ChatGPT / Gemini

"대화창" — 물어보면 답해준다 (사람이 복붙)

Claude Code

"터미널" — 내 컴퓨터 위에서 직접 일한다

환경 세팅 > 프롬프트

흔한 생각

"프롬프트를 잘 쓰면 AI가 잘 답한다"

매번 노력

실제

"환경이 잘 세팅되면, 대충 물어봐도 잘 답한다"

한번 노력 → 매번 좋은 결과

CLAUDE.md

영구 프롬프트

"매번 말 안 해도 아는 맥락"

MCP 연결

자동 입력

"복사 안 해도 AI가 직접 읽는 데이터"

스킬

자동 실행

"한 마디면 전체 프로세스가 돌아감"

이게 OS입니다

"이거 해줘" 한 마디면,

AI가 맥락을 알고 (CLAUDE.md)

데이터를 읽고 (MCP)

프로세스를 실행한다 (스킬)

왜 이 말이 뜨게 됐는가

2023

"프롬프트를 잘 쓰면 된다"

Prompt Engineering

2024

"프롬프트 + 맥락을 잘 구성하면 된다"

RAG, Few-shot

2025~

"맥락 자체를 설계하고 관리해야 한다"

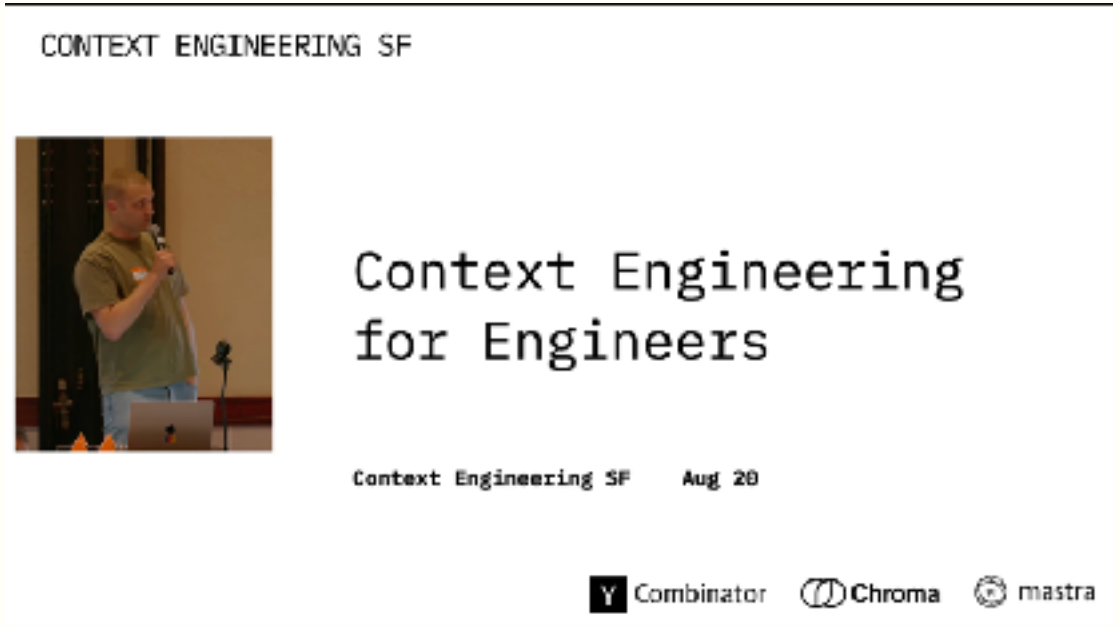
Context Engineering

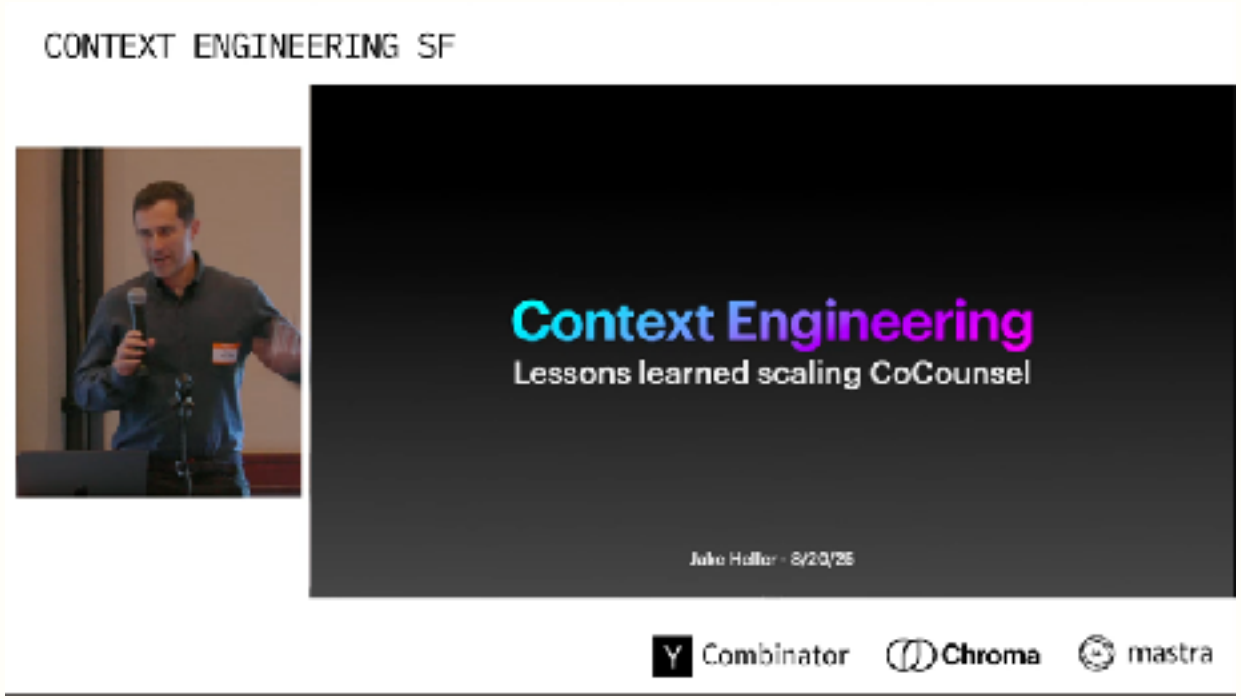
AI 모델 = 순수 함수(Pure Function)

출력의 품질을 결정하는 건 오직 입력의 품질

Garbage In, Garbage Out — 입력 품질 = 출력 품질

지식의 저주 — 왜 맥락 전달이 어려운가





2025년 6월 — 전환점이 된 발언들

Tobi Lutke · Shopify CEO

'prompt engineering'보다 'context engineering'이 훨씬 낫다.

Andrej Karpathy · 전 Tesla AI

컨텍스트 윈도우를 정확히 맞는 정보로 채우는 섬세한 기술이자 과학이다.

Addy Osmani · Google

Prompt engineering이 마법의 한 문장이라면, context engineering은 전체 시나리오를 쓰는 것.

Anthropic · Claude 제작사

"Intelligence is not the bottleneck, context is."

LangChain 2025 (1,340명): 품질 문제의 32%가 AI 능력 부족이 아니라 맥락 관리 실패

지식의 저주 — 왜 맥락 전달이 어려운가

Stanford · Newton 실험 (1990)

120곡을 손가락으로 두드려서 상대방한테 맞히게 함

두드리는 사람 예측

50%

실제 정답률

2.5%

→ 20배 과대평가

MIT Sloan 연구

"전문가는 자기 도메인에서 비전문가보다 더 나쁜 커뮤니케이터다."

IDC 조사

직원 전문 스킬의 42%는 본인만 알고 있음

F500 연간 지식 공유 실패 비용: $31.5B

이 캠프가 처음부터 Context Engineering이었다

Day 1	Day 2	Day 3
CLAUDE.md = **내 맥락을 설계**	**MCP 연결 =** **외부 도구의 맥락을 연결**	**스킬 =** **맥락 기반 워크플로우**
Context 설계	Context 확장	Context 활용

"프롬프트 잘 쓰는 법을 배운 게 아닙니다.

AI가 참고할 맥락 전체를 설계한 겁니다. 이미 하고 있었어요."

업계가 발견한 세 가지 교훈

01

프롬프트가 문제가 아니라 데이터가 문제다

CLAUDE.md가 부정확하면 AI 결과도 부정확

02

계획을 리뷰하는 게 결과를 리뷰하는 것보다 효율적

AI가 만든 2,000줄보다, 200줄 계획을 리뷰하는 게 낫다

03

한번 설계하면 매번 효과

맥락을 한번 잘 설계하면, "대충 물어봐도" AI가 잘 답한다

→ 한번 설계하고 끝이면 낡아간다. 저장소(GitHub) + 반복(Compound Engineering)

이걸 저장하고 관리하는 곳

Google Drive

파일을 저장하고 공유하는 곳

GitHub

저장 + 공유 + 변경 이력 + 협업 리뷰

- 코드는 한 글자만 바꿨는데 전체가 망가질 수 있다
- 그래서 모든 변경을 기록하고, 되돌릴 수 있게 하는 시스템 = Git
- **2025년 기준 1.8억 명이 사용 (GitHub Octoverse)**
- 여러분의 CLAUDE.md, 스킬, MCP 설정도 한 줄 바꾸면 결과가 달라진다
- **바이브 코딩: 코드를 몰라도 AI가 만든다. 근데 망가뜨리면? GitHub이 안전망**

핵심 기능 4가지

Repository

"내 프로젝트의 집"

CLAUDE.md, 스킬 파일, MCP 설정이 사는 곳

Commit

"게임 세이브 포인트"

잘 되는 시점에 저장 + 왜 바꿨는지 메모

Branch

"원본 안 건드리고 실험하는 복사본"

원본(main)은 항상 안전. 새 시도는 브랜치에서

Pull Request

"이렇게 바꿔봤는데 봐줘"

합치기 전 검토 요청. 피드백 → 수정 → 합치기

GitHub = 복리의 인프라

고칠 때마다 commit → 변경 이력이 자동으로 쌓인다

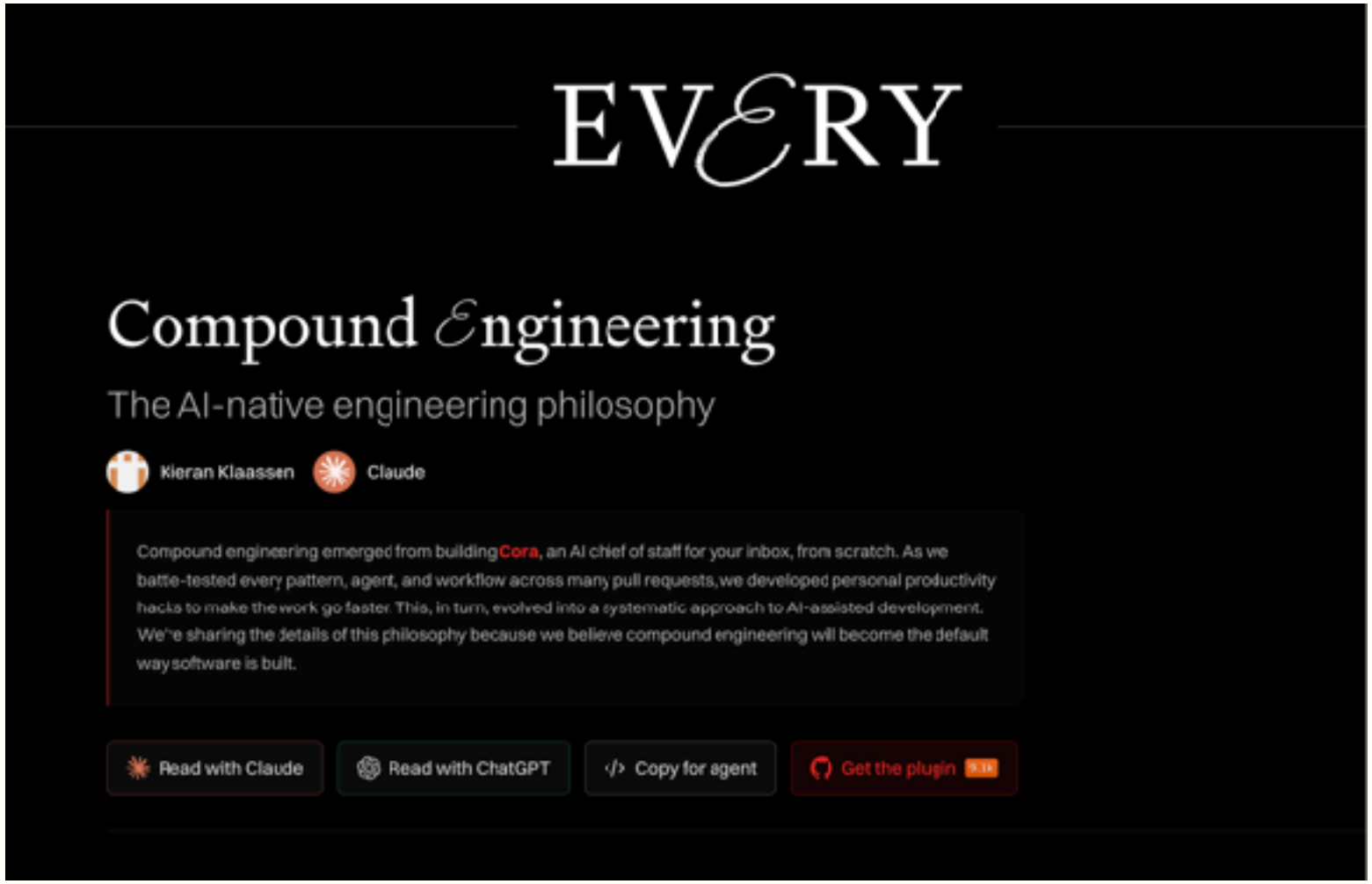
"3일 전 버전이 더 나았네" → 되돌릴 수 있다

기록 없이는 복리가 안 된다

복리의 원리

"AI에게 일을 시킬 때마다 AI를 더 똑똑하게 만들어,

업무 효율을 복리(Compound Interest)처럼 불리는 방법"


EVERY
Compound Engineering
The AI-native engineering philosophy
Kieran Klaassen
Claude
Compound engineering emerged from building Cora, an AI chief of staff for your inbox, from scratch. As we battle-tested every pattern, agent, and workflow across many pull requests, we developed personal productivity hacks to make the work go faster. This, in turn, evolved into a systematic approach to AI-assisted development. We're sharing the details of this philosophy because we believe compound engineering will become the default way software is built.
Read with Claude
Read with ChatGPT
Copy for agent
Get the plugin

The philosophy

The core philosophy of compound engineering is that each unit of engineering work should make subsequent units easier—not harder.

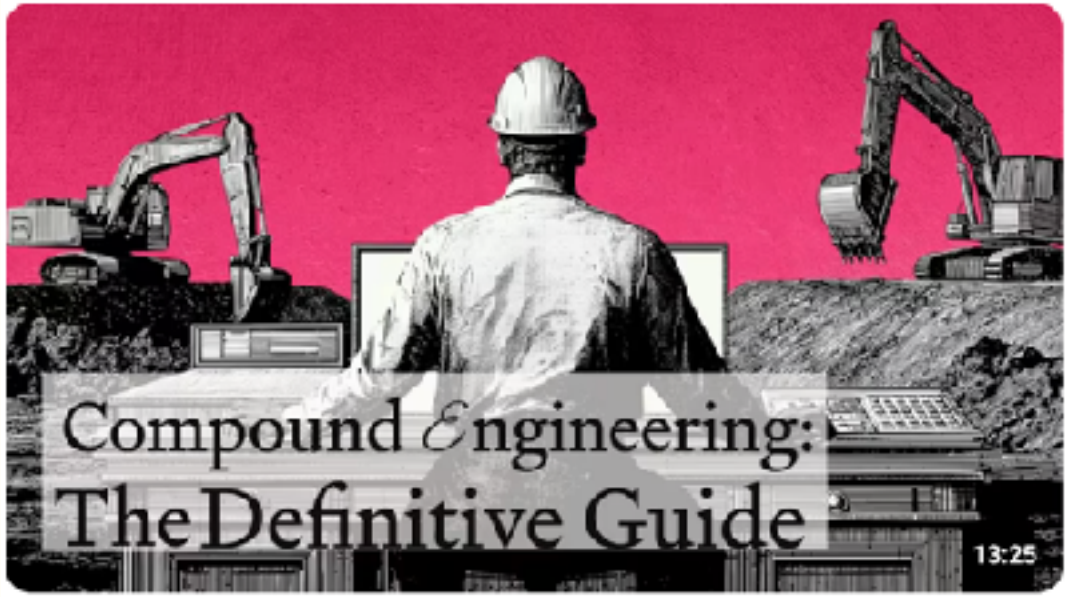
Most codebases get harder to work with over time because each feature you add injects more complexity. After 10 years, teams spend more time fighting their system than building on it because each new feature is a negotiation with the old ones. Over time, the codebase becomes harder to understand, harder to modify, and harder to trust.

Compound engineering flips this on its head. Instead of features adding complexity and fragility, they teach the system new capabilities. Bug fixes eliminate entire categories of future bugs. When they are codified, patterns become tools for future work. Over time, the codebase becomes easier to understand, easier to modify, and easier to trust.

The main loop

Every runs five products—Cora, Monologue, Sparkle, Spiral, and our website Every.to—with primarily single-person engineering teams. The system that makes this possible is a four-step loop that forms the basis of compound engineering:

Plan → Work → Review → Compound → Repeat



Compound Engineering Explained

조회수 4.5천회 • 8일 전

Every

Compound engineering is an AI-native approach to so

✦ 요약 Learn the principles of compound enginee

타이피스트에서 매니저로

과거

직접 타이핑하는 사람

내가 직접 문서 쓰고, 직접 분석하고, 직접 코드 짬

지금

AI를 관리하는 사람

계획을 세워주고, 결과물을 검토하고, 피드백하고 승인

Context Engineering = AI가 참고할 맥락을 설계하는 것 (설계)

Compound Engineering = 그 맥락을 쓰면서 계속 개선하는 것 (반복)

"설계만 하고 안 고치면 낡아갑니다. 고치는 과정이 복리입니다."

"실수 방지 노트 만들기"

기존 방식

AI에게 일 시킴

→ AI가 실수

→ 채팅으로 고쳐달라고 함

→ **다음에 또 같은 실수 반복**

Compound 방식

AI가 실수 → "왜 실수했어?" 원인 파악

→ CLAUDE.md에 규칙 추가

→ **다음부터는 그 실수 안 함**

→ **나만의 완벽한 파트너!**

기존: "잘 만들어서 → 완성하면 → 끝" ✕

우리: "일단 만들고 → 쓰면서 → 계속 고치면 → 복리" ✓

어떻게 쓰느냐 > 어떤 AI를 쓰느냐

Andrew Ng (스탠포드, Google Brain 창립자) · HumanEval 벤치마크

설정	정확도
GPT-3.5 단독	48.1%
GPT-4 단독	67.0%
GPT-3.5 + 에이전틱 워크플로우	95.1%

모델 업그레이드 효과

+19점

워크플로우 효과

+47점

2.5배 더 큰 차이

문서화가 성과를 만든다

Google DORA 2023

질 높은 내부 문서화의 조직 성과 영향

12.8배

DORA 2024 — AI와 문서화

AI가 가장 크게 개선하는 건 코드가 아니라 문서화 품질 (+7.5%)

Anthropic 내부 팀 실제 결과

보안팀 리서치

1시간 → 10~20분

80% 시간 단축

재무팀

개발자 쿼리 요청 → 자연어 자동 생성

개발 의존 제거

법무팀

개발 리소스 요청 → 프로토타입 직접 구축

개발 리소스 불필요

"CLAUDE.md에 워크플로우를 잘 정리한 팀일수록, Claude Code가 더 잘 동작했다."

풀 사이클 — 어떻게 돌리는가

Build

일단 만든다

Use

실제로 써본다

Notice

불편함 발견

Debug

원인을 찾는다

Fix

한 가지만 고친다

Document

기록한다

"한번에 100% 만드는 게 아니라, 1%씩 100번 개선하는 것. 이게 복리입니다."

구체적 이득 3가지

01

같은 말을 반복하지 않아도 된다

"한국어로 답해줘" 매번 말하는 건 비효율. CLAUDE.md에 한번 적으면 영원히 적용.

02

혼자서도 팀만큼의 결과를 낸다

약한 모델 + 좋은 워크플로우(95.1%) > 강한 모델 단독(67.0%). BCG: 품질 +40%

03

미래 핵심 역량이 된다

Microsoft 2025: 글로벌 리더 82%가 AI 에이전트를 팀원으로 도입 예정

Day 1: "AI를 잘 쓰는 건 단계" → Day 4: 그 단계를 반복하는 사이클이 Compound Engineering

"오늘 랩에서 여러분이 할 일은 딱 하나입니다."

지금까지 만든 것 중 하나를 골라서,

써보고, 불편한 점을 찾고, 고쳐보는 것.

이 사이클을 한 바퀴 돌리면,

여러분은 이미 Compound Engineering을 하고 있는 겁니다.

오늘 핵심 3줄

1. 만든 것의 가치는 완성도가 아니라 반복 횟수에 있다 (복리)

2. 불편함은 버그가 아니라 개선의 신호다 (풀 사이클)

3. **사이클을 한 바퀴 돌린 사람은 이미 Compound Engineering을 하고 있다**